the expression `Hotel::Image` is ambiguous because it depends on the execution path.

As we saw before, Ruby looks up the constant in `Hotel` and its ancestors. If `app/models/image.rb`has been loaded but `app/models/hotel/image.rb` hasn't, Ruby does not find `Image` in `Hotel`, but it does in `Object`:

```
$ bin/rails r 'Image; p Hotel::Image' 2>/dev/null
Image # NOT Hotel::Image!
```

The code evaluating `Hotel::Image` needs to make sure `app/models/hotel/image.rb` has been loaded, possibly with `require_dependency`.

In these cases the interpreter issues a warning though:

```
warning: toplevel constant Image referenced by Hotel::Image
```

This surprising constant resolution can be observed with any qualifying class:

```
2.1.5 :001 > String::Array
(irb):1: warning: toplevel constant Array referenced by String::Array
 => Array
```

To find this gotcha the qualifying namespace has to be a class, `Object` is not an ancestor of modules.

10.7 Autoloading within Singleton Classes

Let's suppose we have these class definitions:

```
# app/models/hotel/services.rb
module Hotel
  class Services
  end
end

# app/models/hotel/geo_location.rb
module Hotel
  class GeoLocation
    class << self
      Services
    end
  end
end
```

If `Hotel::Services` is known by the time `app/models/hotel/geo_location.rb` is being loaded, `Services` is resolved by Ruby because `Hotel` belongs to the nesting when the singleton class of `Hotel::GeoLocation` is opened.

But if `Hotel::Services` is not known, Rails is not able to autoload it, the application raises `NameError`.

The reason is that autoloading is triggered for the singleton class, which is anonymous, and as we saw before, Rails only checks the top-level namespace in that edge case.

An easy solution to this caveat is to qualify the constant:

```
module Hotel
  class GeoLocation
    class << self
      Hotel::Services
    end
  end
end
```

10.8 Autoloading in `BasicObject`

Direct descendants of `BasicObject` do not have `Object` among their ancestors and cannot resolve top-level constants:

```
class C < BasicObject
  String # NameError: uninitialized constant C::String
```